

DIVIDE AND CONQUER

INTRODUCTION



**SCHOOL OF
INFORMATION TECHNOLOGY**



IT IS WIDELY USED IN.....

- Sorting algorithms (Merge Sort, Quick Sort)
- Searching
- Computational geometry
- Parallel computing



DEFINITION

- **Divide and Conquer is a strategy where a problem is:**
 - Divided into smaller subproblems
 - Solved recursively
 - Combined to form the final solution





THREE STEPS OF DIVIDE AND CONQUER

- **Step 1: Divide**
 - Split the problem into smaller subproblems
 - Subproblems should be of the same type
- **Example:**
 - Split an array into two halves





THREE STEPS OF DIVIDE AND CONQUER

- **Step 2: Conquer**
 - Solve each subproblem (usually recursively)
 - Stop when you reach the base case
- **Base Case:**
 - Simple problem that can be solved directly





THREE STEPS OF DIVIDE AND CONQUER

- **Step 3: Combine**
 - Merge or combine the solutions of subproblems
 - Produce the final solution





GENERAL STRUCTURE (RECURRENCE)

- **A divide and conquer algorithm typically follows:**

- $T(n) = aT(n/b) + f(n)$

- **Where:**

- a = number of subproblems
 - n/b = size of each subproblem
 - $f(n)$ = cost of dividing + combining





WHAT IS A RECURRENCE RELATION?

- Is a mathematical expression that defines the running time of an algorithm in terms of smaller inputs of the same problem.





GENERAL STRUCTURE (RECURRENCE)

- **General Recurrence Formula**

- $T(n) = aT(n/b) + f(n)$

- **a: Number of Subproblems**

- How many smaller problems we create

- **Example:**

- Merge Sort → splits into 2 subproblems → $a=2$





GENERAL STRUCTURE (RECURRENCE)

- **General Recurrence Formula**

- $T(n) = aT(n/b) + f(n)$

- **Each subproblem is smaller than the original**

- **Example:**

- If array is split into halves →
new size = $n/2$





GENERAL STRUCTURE (RECURRENCE)

- **General Recurrence Formula**

- $T(n) = aT(n/b) + f(n)$

- **$f(n)$: Work Outside Recursion**

- **This includes:**

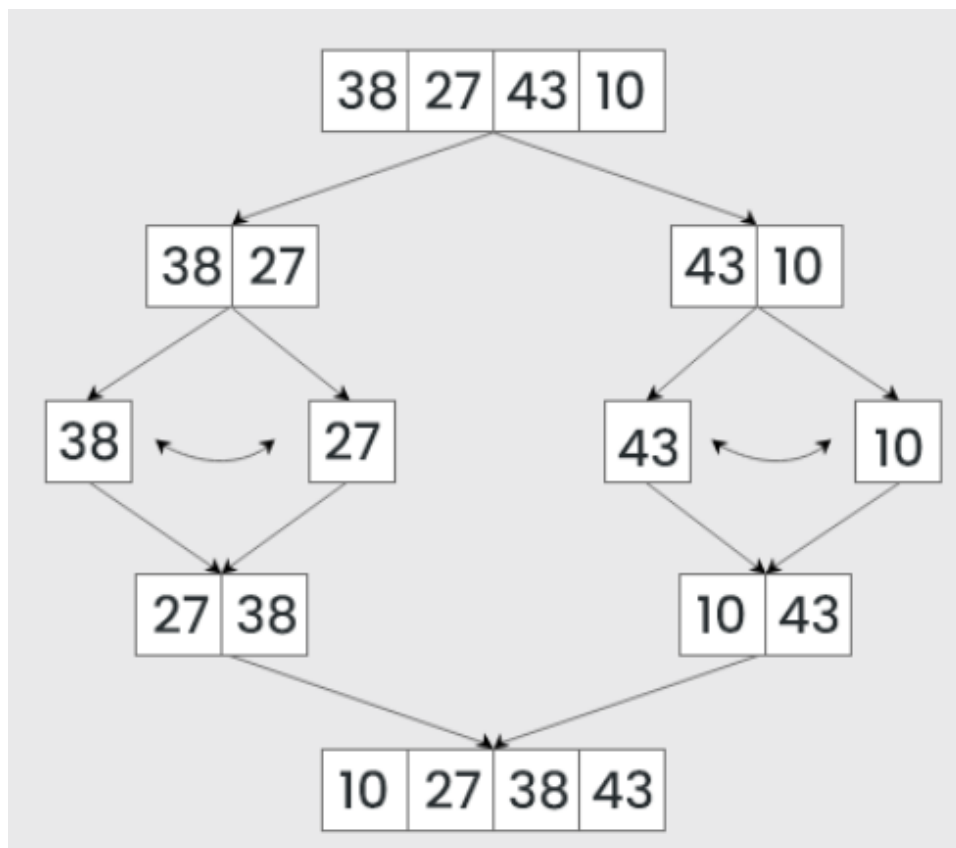
- Dividing the problem
 - Combining results

- **Example:**

- Merge Sort:
 - Merging takes $O(n) \rightarrow$ so $f(n) = n$



EXAMPLE BREAKDOWN: MERGE SORT



EXAMPLE BREAKDOWN: MERGE SORT

- **Step-by-step recurrence:**

- $T(n) = 2T(n/2) + n$

- $T(n) = 2T(n/2) + n$ [1]

- **subst: $n/2$**

- $= 2[2T((n/2)/2) + n/2] + n$ [2]

- $= 4T(n/4) + n + n$

- $= 4T(n/4) + 2n$

- $= 4[2T((n/2)/4) + (n/4)] + 2n$

- $= 8T(n/8) + 3n$

- ...

- $= nT(1) + n \log n$

- $= O(n \log n)$



EXAMPLE BREAKDOWN: MERGE SORT

- **Step-by-step recurrence:**

- $T(n) = 2T(n/2) + n$

- $8T(n/8) + 3n$

- **After k steps:**

- $T(n) = 2^k T(n/2^k) + kn$

- **Reaching the Base Case**

- We stop when:

- $n/2^k = 1$

- Solve for k:

- $2^k = n \Rightarrow k = \log_2 n$



EXAMPLE BREAKDOWN: MERGE SORT

- **Step-by-step recurrence:**

- $T(n) = 2T(n/2) + n$

- **Reaching the Base Case**

- We stop when:

- $n/2^k = 1$

- Solve for k:

- $2^k = n \Rightarrow k = \log_2 n$

- **At base case:**

- $T(n) = 2^{\log n} T(1) + n \log n$

- **Simplify:**

- $2^{\log n} = n$

- **So:**

- $T(n) = nT(1) + n \log n$





EXAMPLE BREAKDOWN: MERGE SORT

- **Step-by-step recurrence:**

- $T(n) = 2T(n/2) + n$

- $T(n) = nT(1) + n \log n$

- **Since $T(1)$ is constant:**

- $T(n) = O(n) + O(n \log n)$

- **FINAL RESULT**

- $T(n) = O(n \log n)$





EXAMPLE BREAKDOWN: MERGE SORT (SUMMARY)

- **Each Level:**
 - You process n elements total
- **Number of Levels:**
 - You divide the problem until size becomes 1
 - Number of levels = $\log n$
- **Total Work:**
 - Work per level $\times$ levels
 - $= n \times \log n$





EXAMPLE BREAKDOWN: MERGE SORT (SUMMARY)

- **Why This is Efficient?**
 - Compared to brute force:
 - $O(n^2) \rightarrow$ very slow
 - $O(n \log n) \rightarrow$ much faster



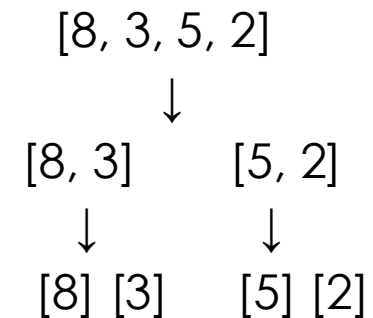
MERGE SORT

- **Problem**

- Sort the array:
 - [8, 3, 5, 2]

- **Step 1: Divide**

- We repeatedly divide the array into smaller subarrays.



- Stop dividing when each subarray has only 1 element
- This is the base case





MERGE SORT

- **Problem**

- Sort the array:
 - [8, 3, 5, 2]

- **Step 2: Conquer (Sort Small Parts)**

- Single elements are already sorted:
 - [8] [3] [5] [2]





MERGE SORT

- **Problem**

- Sort the array:
 - [8, 3, 5, 2]

- **Step 3: Combine (Merge Step-by-Step)**

- **Merge [8] and [3]**

- Compare: 8 vs 3
- 3 is smaller

- **Result: [3, 8]**





MERGE SORT

- **Problem**

- Sort the array:
 - [8, 3, 5, 2]

- **Step 3: Combine (Merge Step-by-Step)**

- **Compare:**

- 5 vs 2
- 2 is smaller

- **Result:[2, 5]**





MERGE SORT

- **Problem**

- Sort the array:
 - [8, 3, 5, 2]

- **Final merged array:**

- [2, 3, 5, 8]

- **Step 3: Combine (Merge Step-by-Step)**
- **Final Merge: [3, 8] and [2, 5]**
- **Compare step by step:**

Comparison	Result
3 vs 2	2
3 vs 5	3
8 vs 5	5
remaining	8





ACTIVITY

- **Given:**
 - [8, 3, 5, 2, 7, 1, 4, 6]
- **Students:**
 - Perform merge sort
 - For each level, count:
 - Number of comparisons
 - Total elements processed

✓ Table Format:

Level	Subarrays	Work Done
1	1 array	n
2	2 arrays	n
3	4 arrays	n



END



SCHOOL OF
INFORMATION TECHNOLOGY